

A Comparison of Neural Recommender System Architectures

Ken Low

University of Colorado, Boulder

ABSTRACT

Recommender systems take information about user and item characteristics, learn meaningful patterns and relationships, and then determine the relevance of a given item for a target user. These relationships are often represented through interactions among users, items, and observed links, which makes graph-based formulations useful for recommendation problems. Neural networks are well suited to this setting because they can model complex, nonlinear relationships. This study compares several neural recommender architectures, hyperparameter-search methods, and optimizer families within a lightweight experimental harness. Applying a consistent, small training budget across different runs, the results indicate the following: Simpler embedding-based models and explicit relational features were often highly competitive, grid search produced the best hyperparameter-search outcomes, and gradient-based optimization methods outperformed population-based alternatives. These findings suggest that model design, feature selection, and optimization strategy all materially affect recommender-system performance.

1 INTRODUCTION

Recommender systems are widely used in many applications such as movie recommendations to users on entertainment platforms, featured products on online commerce sites, and even friend suggestions on social networks. To make effective recommendations that align with users' preferences, a recommender system must learn about the characteristics of different users and items and then infer which items to match with a specific user. Machine learning in recommender systems is well suited to graph-based representations because data on users and items can be mapped naturally to a graph structure, and graphs can capture complex, nonlinear relationships (Wang et al., 2021). In addition, neural network modeling has become more popular because of its expressive power and its ability to capture more complex relationships, which makes neural methods a natural choice for recommender-system development (Gridach, 2020).

2 RELATED WORK

Neural recommender systems have been widely studied as a way to model user-item interactions with learned distributed representations rather than manually engineered similarity rules alone. Embedding-based deep neural recommenders treat users and items as trainable vectors and then score candidate interactions with feedforward networks, as in neural collaborative filtering approaches such as He et al. (2017). This line of work motivates the non-graph Deep Neural Network (DNN) baseline in the present study, where recommendation is framed as an edge-prediction problem over learned node embeddings.

Graph neural networks extend this idea by allowing node representations to absorb information from neighboring nodes and edges. In recommender settings, graph-based learning has been used to model user-item interactions more explicitly than plain embedding methods, especially when interactions can be represented as edges in graphs, as shown by neural graph collaborative filtering and subsequent simplified graph recommenders (Wang et al., 2019; He et al., 2020). This literature motivates the inclusion of a graph neural network baseline in the study and provides the basis for comparing graph-aware and non-graph neural recommenders under a shared experimental protocol.

A second relevant body of work concerns hybrid recommender architectures that introduce additional inductive biases into neural models. Sequence-aware recommenders use recurrent networks such as gated recurrent units (GRUs) and long short-term memory (LSTM) models to capture temporal order in user behavior, as in session-based recommendation models such as Hidasi et al. (2016), while graph-and-sequence hybrids combine structural information from interaction graphs with temporal summaries of past interactions. Related efforts have also explored combining neural recommenders with hidden-state or regime-switching ideas, which motivates the RHMM-inspired variant examined in this study (Djellali & Adda, 2020).

Another important thread in the literature augments neural recommendation with relational rules or symbolic structure. Probabilistic Soft Logic (PSL) and related neuro-symbolic approaches show that handcrafted relational signals, such as shared neighbors or degree-based patterns, can complement learned latent representations (Bach et

al., 2017). The PSL-DNN model in the present study follows this general direction, although it should be understood as a lightweight PSL-style approximation based on graph heuristics rather than a full probabilistic logic inference system.

Reinforcement learning (RL) has also influenced recommender-system research, especially in settings where a model must adapt decisions over time or choose among competing strategies. In addition to full RL formulations for recommendation, lighter policy-based mechanisms have been used to control components of neural architectures (Zou et al., 2019). This body of work motivates the RL-guided aggregation variant in the current study, where a bandit-style policy selects among alternative graph-aggregation strategies during training.

Beyond model architecture, prior work has shown that hyperparameter selection and optimization strategy can materially affect neural-network performance. Grid search and random search remain standard baselines for hyperparameter tuning, with Bergstra and Bengio (2012) showing the practical strength of random search, while population-based search procedures such as Differential Evolution provide an alternative for exploring mixed continuous and discrete search spaces (Storn & Price, 1997). Similarly, gradient-based optimizers such as stochastic gradient descent and Adam are central to modern deep learning (Kingma & Ba, 2014), whereas population-based training methods such as particle swarm optimization and evolutionary strategies provide gradient-free alternatives (Kennedy & Eberhart, 1995). These strands of research motivate the study's additional comparisons of hyperparameter-search methods and optimizer families.

Taken together, the prior literature suggests that recommender performance depends not only on whether a model is neural, graph-based, sequential, symbolic, or policy-guided, but also on how the model is tuned and optimized. Much of this prior work, however, evaluates these issues in isolation. The present study addresses that gap by comparing model families, hyperparameter-search strategies, and optimizer classes within a single experimental harness on recommendation-style edge-prediction tasks.

3 METHODOLOGY

The goal of the experiment is to compare neural recommender variants under a shared experimental harness. For each run, predictive performance, wall-clock time, and resource usage are recorded and compared.

3.1 Datasets

The MovieLens 100K dataset is a user-item rating table and serves as the smaller benchmark for rapid validation of the experimental pipeline. The ogbl-collab dataset from the Open Graph Benchmark (OGB) is a large author-collaboration graph in which nodes denote authors and edges denote collaborations. Together, these datasets provide a contrast between a smaller implicit-feedback recommendation setting and a larger graph-learning benchmark.

3.2 Data Transformation

The datasets are converted into edge-prediction tasks. MovieLens 100K is transformed into a bipartite user-item graph, while ogbl-collab remains a homogeneous collaboration graph. Positive edges come from observed interactions, and negative edges are sampled from unobserved node pairs rather than treating every missing edge as an explicit negative label.

In the MovieLens 100K dataset, ratings of 4 and 5 are treated as positive interactions. Ratings below 4 are excluded so that the task becomes implicit-feedback link prediction instead of explicit rating regression.

For the Probabilistic Soft Logic-style deep neural network (PSL-DNN) model, additional graph-heuristic features are computed from the training graph, including source and destination degrees, a degree-product term, the number of shared neighbors, and overlap ratios between neighbor sets. These features act as a lightweight proxy for Probabilistic Soft Logic (PSL)-style relational evidence rather than a full PSL inference engine.

3.3 Model Comparisons

The recommender models were compared along three axes: model family, hyperparameter search strategy, and optimization method.

3.3.1 Model-Family Comparison

The first suite of experiments compared different neural recommender architectures. Each model family was trained under the same default hyperparameters and training-budget settings so that the primary source of variation was architectural rather than procedural. The comparison begins with two baselines: a non-graph deep neural network (DNN) that scores candidate node pairs from learned embeddings, and a graph neural network (GNN) that augments those embeddings with neighborhood information from the training graph.

3.3.2 Deep Neural Network (DNN)

The non-graph baseline recommender is implemented with PyTorch's `EmbeddingMLPRecommender` class. For each mini-batch, the model assigns an embedding to each entity such as a user or item, constructs pairwise features from the source and destination embeddings, and passes those features to a multilayer perceptron (MLP) scoring head. The MLP scoring head consists of the following components:

- a linear layer
- the rectified linear unit (ReLU) function
- the Dropout regularization method, which randomly turns off a fraction of layer activations during training only
- followed by the final linear layer that gives the output logit score

In this study, the DNN scoring head uses only two linear layers because network depth is not the main variable of interest and a shallow architecture keeps the baseline computationally simple. The predicted logit for each sampled edge is compared with its binary label, and backpropagation updates both the embedding table and the scoring-head weights to reduce binary cross-entropy loss.

3.3.3 Graph Neural Network (GNN)

The graph baseline recommender is implemented with PyTorch's `GNNRecommender` class. Each graph node's embedding is updated by aggregating one-hop neighbor information over the training graph using mean message aggregation. The resulting graph-aware node representations are then combined into pairwise features and passed to the same MLP scoring head used in the DNN baseline. The graph neural network consists of the following layers and steps:

- Two GraphEncoder layers, each of which applies:
 - A linear layer to update the graph node's current representation
 - Another linear layer to update the aggregated neighbor representation
 - ReLU
 - Dropout for regularization
- A fixed combination of the initial embeddings and graph-layer outputs (mean in the baseline GNN)
- Computing feature vectors representing each pair of graph nodes in the training sample
- An MLP classifier to predict the presence of an edge for each pair, consisting of:
 - A linear layer
 - ReLU
 - Dropout for regularization
 - Another linear layer that outputs the logit for whether the edge should exist

The predicted logits are compared with sampled labels using binary cross-entropy loss, and the optimizer updates the trainable parameters to minimize that loss.

3.3.4 Hybrid Hidden Markov-Neural Network

This hybrid neural network is inspired by RHMM-style sequential recommendation (Djellali & Adda, 2020), but it is not a full hidden Markov model implementation. For each candidate pair of nodes, the model extracts the source node's recent interaction history in temporal order, encodes that sequence with a single-layer gated recurrent unit (GRU), and then infers a soft latent regime mixture from the GRU state – a probability-weighted mixture of several possible hidden behavioral states of the source node. The GRU summary, regime vector, and destination-node embedding are concatenated and scored with the same MLP head used elsewhere. The significance is that the temporal order of previous interactions is explicitly modeled instead of being ignored.

3.3.5 Hybrid Probabilistic Soft Logic (PSL) Deep Neural Network

This hybrid model augments the embedding-based DNN with PSL-like relational features. In practice, the implementation appends handcrafted graph statistics such as source and destination degree, the degree-product term, and neighbor-overlap ratios to the learned pairwise embedding features, then scores the combined vector with the MLP classifier. The model is therefore PSL-like in the sense that it injects soft relational evidence, not because it runs an external PSL solver.

3.3.6 Hybrid Bidirectional Long Short-Term Memory (LSTM) Graph Neural Network

This graph-neural variant uses a bidirectional LSTM to summarize each source node's recent sequence of historical interactions, then combines that sequence summary with graph-aware source and destination embeddings produced by the GNN encoder. The resulting feature vector is scored by the MLP head to estimate whether the candidate edge should exist.

3.3.7 Hybrid RL-Optimized GNN Aggregation

This variant keeps the same GNN encoder and MLP scorer but adds a lightweight bandit-style policy over several ways of combining graph-layer outputs (last, mean, and a residual-style weighted combination). During training, the policy samples an aggregation expert, receives reward based on the batch loss, and updates its preference logits. At evaluation time, the expert with the strongest learned policy preference is used.

3.4 Comparison of Hyperparameter Search Methods

The second suite of experiments was run with the goal of comparing the performance of different hyperparameter search methods. Grid search, random search, and Differential Evolution (DE)-style search methods were tested. The hyperparameters tuned were the length of the graph node embeddings, the size of hidden layers, the dropout fraction, and the learning rate. Each hyperparameter candidate was evaluated by training the model for a short run.

Grid search evaluates every combination in a predefined discrete hyperparameter grid. Random search samples hyperparameter combinations independently from the predefined search space, providing a less exhaustive but cheaper alternative to grid search. In DE-style search, a small population of candidate hyperparameter vectors is initialized, evaluated, and then iteratively mutated and crossed over using differences between other candidates in the population. A trial candidate replaces the incumbent when it improves validation area under the curve (AUC). The search runs for a fixed number of generations rather than until a guaranteed optimum is found.

3.5 Optimizer Comparison: Gradient-Based vs. Population-Based Training

The third suite of experiments compared gradient-based optimizers with population-based alternatives. Adam and stochastic gradient descent (with momentum 0.9) were the gradient methods, while particle swarm optimization (PSO) and an evolution-strategy-style optimizer were the population-based methods. The optimizer comparison used a dataset-appropriate baseline in each case: a DNN on MovieLens 100K and a GNN on ogbl-collab.

Adam is one of the most frequently cited optimization algorithms for neural networks, adapting the learning rate with estimates of the first and second moments of the gradients (Kingma & Ba, 2014). SGD computes the gradient from a mini-batch of pairs of graph nodes. In PSO, the model generates multiple sets of parameters and represents each set as a particle; personal and global best values guide updates until the particles converge. Evolutionary strategy is a population-based gradient-free optimizer that iteratively perturbs a parameter vector and shifts it toward perturbations that reduce loss.

3.6 Training Budget

The experimental harness was intentionally lightweight because of limited system resources. The host machine had eight CPU cores and 16 GB of RAM. Gradient-based runs used 24 – 175 stochastic optimizer updates, depending on the dataset and experiment suite, with each update using a newly sampled batch of 512 positive and 512 negative edges. In the optimizer comparison, population-based methods evaluated candidate parameter vectors against a fixed batch of 256 positive and 256 negative edges. PSO performed 130 – 210 objective evaluations per run, while evolutionary training performed 191 – 343. The smaller population batch partially balanced aggregate evaluation workload: Adam and SGD scored approximately 61,440 – 179,200 edge examples per run, compared with 66,560 – 175,616 for the population-based methods, while limiting the multiplicative cost of evaluating 10 – 18 parameter vectors per iteration. It should be noted that these totals do not represent equivalent unique-data exposure because gradient batches were resampled between updates, whereas population candidates repeatedly used the same fixed fitness batch. The protocol therefore approximated total evaluation workload rather than enforcing identical batch sizes or identical data exposure.

4 RESULTS

Tables 1–3 summarize the results by predictive performance, runtime, and compute-resource usage. Higher Test AUC, Test AP, and accuracy indicate better predictive performance, whereas lower runtime and memory use indicate greater efficiency.

4.1 Model-Family Comparison

Dataset	Model	Method	Test AUC	Test AP	Accuracy	Time (s)	Peak RAM (MB)	Peak CPU %
movielens_100k	dnn	adam	0.5244	0.5047	0.4810	0.4208	699.4	833.3
movielens_100k	psl_dnn	adam	0.4716	0.4773	0.4829	2.0471	740.0	1562.5
movielens_100k	rhmm	adam	0.4631	0.4707	0.4771	4.0528	739.4	1666.7
movielens_100k	gnn_bilstm	adam	0.4582	0.4706	0.4814	6.1879	767.9	1209.7
movielens_100k	gnn	adam	0.4484	0.4659	0.4808	2.3671	718.0	1666.7
movielens_100k	rl_gnn	bandit_policy	0.3315	0.4165	0.4665	2.4957	762.1	1562.5
ogbl_collab	psl_dnn	adam	0.9304	0.9356	0.8338	4.2315	1547.7	1063.8
ogbl_collab	dnn	adam	0.9222	0.9220	0.8327	2.3012	1220.7	1209.7
ogbl_collab	gnn_bilstm	adam	0.7886	0.8105	0.6888	22.7506	1562.9	783.4
ogbl_collab	rl_gnn	bandit_policy	0.7676	0.8063	0.7188	20.0864	1567.3	781.3
ogbl_collab	gnn	adam	0.7443	0.7982	0.6908	19.2167	1232.6	781.3
ogbl_collab	rhmm	adam	0.7136	0.7713	0.7286	5.8375	1260.4	961.5

Table 1. Model-family comparison across datasets.

Across the model-family comparison, predictive performance varied sharply by dataset. On MovieLens 100K, the baseline DNN achieved the best held-out AUC while also remaining among the fastest and most memory-efficient models, indicating that the additional complexity of the hybrid variants was not justified under the lighter training budget. On ogbl-collab, however, PSL-DNN produced the strongest predictive results, while the plain DNN remained highly competitive and substantially faster than the more complex sequence-aware variants. The graph-sequential and RL-guided models generally consumed more time and memory without exceeding the strongest simpler baselines. Notably, in the context of this experiment, embedding-based DNNs outperformed GNNs when trained and tested on both datasets.

4.2 Hyperparameter-Search Comparison

Dataset	Search Method	Best AUC	Best AP	Best Acc.	Avg Time (s)	Fastest (s)	Avg RAM (MB)	Low RAM (MB)	Best Config / Notes
movielens_100k	grid_search	0.7107	0.6616	0.6485	0.1808	0.1464	691.9	691.7	emb=16, hid=64, drop=0.0, lr=0.001
movielens_100k	de_opt	0.6817	0.6217	0.5870	0.2173	0.1867	699.1	698.8	DE-style search
movielens_100k	random_search	0.6287	0.5753	0.5260	0.2004	0.1515	696.7	692.6	emb=32, hid=64, drop=0.1, lr=0.001
ogbl_collab	grid_search	0.7792	0.7927	0.6888	8.6597	5.3999	828.2	712.9	emb=32, hid=32, drop=0.2, lr=0.001
ogbl_collab	random_search	0.7690	0.8070	0.6892	10.9093	6.9820	1072.1	801.9	emb=16, hid=32, drop=0.2, lr=0.01
ogbl_collab	de_opt	0.7595	0.8000	0.6888	12.8570	6.8480	1234.3	1224.3	DE-style search

Table 2. Hyperparameter-search comparison using best predictive result per search method, plus time and memory summaries.

The hyperparameter-search comparison results indicate that grid search outperformed random search and DE-style search. Among the three options, it consistently achieved the highest AUC and accuracy and the shortest runtime and lowest memory usage across both datasets. It is interesting that random search, despite fewer iterations than grid search, still took longer to complete and consumed more memory. The DE-style search's more adaptive procedures did not translate the extra search complexity into better final model quality.

4.3 Optimizer Comparison

Dataset	Model	Optimizer	Test AUC	Test AP	Accuracy	Time (s)	Peak RAM (MB)	Peak CPU %
movielens_100k	dnn	pso	0.5200	0.5307	0.5102	0.5341	707.9	1562.5
movielens_100k	dnn	evolutionary	0.5052	0.5061	0.5000	0.4149	703.7	1666.7
movielens_100k	dnn	sgd	0.4999	0.4880	0.4925	0.5087	691.0	833.3
movielens_100k	dnn	adam	0.4812	0.4809	0.4864	0.6456	690.2	833.3

Dataset	Model	Optimizer	Test AUC	Test AP	Accuracy	Time (s)	Peak RAM (MB)	Peak CPU %
ogbl_collab	gnn	adam	0.7299	0.7859	0.7222	23.1336	1235.2	771.2
ogbl_collab	gnn	pso	0.6720	0.6559	0.6558	38.5761	2339.9	718.9
ogbl_collab	gnn	evolutionary	0.6630	0.6234	0.6405	38.8448	1956.7	687.3
ogbl_collab	gnn	sgd	0.6574	0.5676	0.5014	22.7986	1240.5	781.2

Table 3. Optimizer comparison across datasets.

On MovieLens 100K, PSO achieved the highest test AUC, but the performance gaps among optimizers were small. On ogbl-collab, Adam provided the best predictive performance while also using less memory than the population-based alternatives, making it the strongest overall choice. Population-based methods were slower and more memory intensive, whereas SGD was slightly faster than Adam but substantially weaker in predictive quality on the larger benchmark.

Overall, the recorded runs indicate that lightweight baseline models and explicit PSL-style relational features were highly competitive in this experimental setup, whereas more complex hybrid architectures and population-based optimization methods were harder to justify under the conservative compute budget.

5 CONCLUSION

Within the bounds of this lightweight experimental harness, the study shows that when comparing different families of models, simpler embedding-based models and explicit relational features were often more effective than more elaborate hybrid variants, especially under short training budgets. Across the hyperparameter-search comparison, grid search was the most reliable strategy in the tested discrete search spaces. As for optimizers, gradient-based methods, especially Adam on the larger graph benchmark, offered the strongest balance of predictive performance and computational efficiency. The results suggest that careful feature design, controlled tuning, and appropriate optimizer selection can matter as much as architectural complexity in practical neural recommender experiments.

6 FUTURE WORK

There are several promising directions for future work beyond the scope of this study. First, the experiments could be repeated with larger training budgets, multiple random seeds, and confidence intervals so that the observed differences can be evaluated more robustly. Second, future studies could include additional recommender-system metrics such as normalized discounted cumulative gain (NDCG)@K, Recall@K, and Hit Rate@K alongside the current edge-prediction measures. Third, the hybrid variants could be implemented in more faithful form, including richer RHMM-style sequence modeling, tighter probabilistic soft logic integration, and more sophisticated reinforcement-learning-based control. Finally, it would be valuable to examine how these methods behave in settings that more closely resemble production recommender systems, including larger-scale data, repeated retraining, and stronger ranking-focused evaluation protocols.

REFERENCES

- Bach, S. H., Broecheler, M., Huang, B., & Getoor, L. (2017). Hinge-loss Markov random fields and probabilistic soft logic. *Journal of Machine Learning Research*, 18(109), 1–67. <https://jmlr.org/papers/v18/15-631.html>
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305. <https://www.jmlr.org/papers/v13/bergstra12a.html>
- Djellali, C., & Adda, M. (2020). A new hybrid deep learning model based-recommender system using artificial neural network and hidden Markov model. *Procedia Computer Science*, 175, 214–220. <https://doi.org/10.1016/j.procs.2020.07.032>
- Gridach, M. (2020). Hybrid deep neural networks for recommender systems. *Neurocomputing*, 413, 23–30. <https://doi.org/10.1016/j.neucom.2020.06.025>
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web* (pp. 173–182). <https://doi.org/10.1145/3038912.3052569>
- He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 639–648). <https://doi.org/10.1145/3397271.3401063>
- Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). Session-based recommendations with recurrent neural networks. *arXiv*. <https://doi.org/10.48550/arXiv.1511.06939>

- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In Proceedings of ICNN'95 - International Conference on Neural Networks (pp. 1942–1948). <https://doi.org/10.1109/ICNN.1995.488968>
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv. <https://doi.org/10.48550/arXiv.1412.6980>
- Storn, R., & Price, K. (1997). Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359. <https://doi.org/10.1023/A:1008202821328>
- Wang, X., He, X., Wang, M., Feng, F., & Chua, T.-S. (2019). Neural graph collaborative filtering. In Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 165–174). <https://doi.org/10.1145/3331184.3331267>
- Wang, S., Hu, L., Wang, Y., He, X., Sheng, Q. Z., Orgun, M. A., Cao, L., Ricci, F., & Yu, P. S. (2021). Graph learning based recommender systems: A review. arXiv. <https://doi.org/10.48550/arXiv.2105.06339>
- Zou, L., Xia, L., Ding, Z., Song, J., Liu, W., & Yin, D. (2019). Reinforcement learning to optimize long-term user engagement in recommender systems. In Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 2810–2818). <https://doi.org/10.1145/3292500.3330668>